

Tema 4.3.4: TypeScript Avanzado, Genéricos, Tipos de Utilidad y Decoradores

Resumen

Este tema aborda las características avanzadas del sistema de tipos de **TypeScript**, esenciales para el desarrollo de frameworks modernos como Angular. Se examina la programación con Genéricos (*Generics*), la composición mediante tipos de Unión e Intersección, el estrechamiento de tipos (*Type Narrowing*) y *Type Guards*, la manipulación de tipos mediante los *Utility Types* incorporados (`Partial`, `Pick`, `Omit`, `Record`), y el uso de Decoradores para metaprogramación declarativa.

Concepto Principal (El 'Qué')

1. Genéricos (*Generics* <T>)

Los **Genéricos** permiten crear componentes, clases e interfaces reutilizables que funcionan sobre una variedad de tipos de datos en lugar de un único tipo concreto, manteniendo el 100% del rigor del tipado estático en tiempo de compilación.

```
// Interface Genérica para Respuestas Paginadas de API REST
export interface RespuestaPaginada<T> {
  contenido: T[];
  totalElementos: number;
  paginaActual: number;
  totalPaginas: number;
}
```

- **Restricciones en Genéricos (*extends*):** Permiten limitar los tipos que pueden pasarse como parámetro genérico (ejemplo: `<T extends { id: number }>`).

2. Estrechamiento de Tipos (*Type Narrowing*) y *Type Guards*

Cuando una variable se declara como un Tipo de Unión (ejemplo: `string | number`), TypeScript exige "estrechar" su tipo antes de invocar métodos específicos de una de las opciones:

- **Operadores `typeof` e `instanceof`**: Evalúan tipos primitivos o instancias de clases en tiempo de ejecución.
- **Predicados de Tipo Personales (`is`)**: Funciones especiales de aserción que retornan un booleano e informan al compilador de TypeScript sobre la confirmación del tipo.

3. Tipos de Utilidad de TypeScript (*Utility Types*)

TypeScript incluye tipos globales incorporados para transformar tipos existentes sin duplicar código:

- **`Partial<T>`**: Convierte todas las propiedades de `T` en opcionales (`?`). Ideal para payloads de actualización parcial HTTP PATCH.
- **`Required<T>`**: Convierte todas las propiedades opcionales de `T` en obligatorias.
- **`Readonly<T>`**: Hace que todas las propiedades de `T` sean inmutables de sólo lectura.
- **`Pick<T, K>`**: Crea un nuevo tipo seleccionando un subconjunto de claves `K` de `T`.
- **`Omit<T, K>`**: Crea un nuevo tipo omitiendo un subconjunto de claves `K` de `T`. Ideal para DTOs de creación que no incluyen el `id` autogenerado.
- **`Record<K, T>`**: Construye un tipo de objeto cuyas claves son `K` y sus valores son `T` (equivalente a diccionarios o mapas tipados).

4. Decoradores (Metaprogramación para Frameworks)

Los **Decoradores** son anotaciones especiales (precedidas por `@`) que se adjuntan a clases, métodos, propiedades o parámetros para agregar comportamiento o metadatos de forma declarativa. Constituyen la columna vertebral de la arquitectura modular de **Angular** (`@Component`, `@Injectable`, `@Input`).

Analogía (La Intuición)

Imagine un contenedor marítimo estandarizado internacional frente a moldes de inyección de plástico:

- **Genéricos <T> (El Contenedor Marítimo Estandarizado):** La grúa del puerto y el barco carguero están diseñados para mover contenedores estandarizados sin importar si por dentro el contenedor lleva vehículos (`RespuestaPaginada<Vehiculo>`), computadoras (`RespuestaPaginada<Computadora>`) o alimentos (`RespuestaPaginada<Alimento>`). La grúa (la función genérica) garantiza el transporte seguro sin romper el sello de carga (manteniendo el tipado estático).
- **Utility Types (`Omit`, `Partial`) (Moldes de Inyección de Plástico):** Si usted tiene el molde maestro de una silla completa (`vehiculo`), y requiere fabricar un modelo sin brazos (`Omit<Vehiculo, 'brazos'>`) o un modelo plegable ajustable (`Partial<Vehiculo>`), no dibuja un plano nuevo desde cero. Toma el molde maestro existente y le aplica una modificación industrial directa.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se muestra un servicio genérico asíncrono con *Utility Types*, *Type Guards* y Decoradores:

```
import { VehiculoDto } from './modelos';

// 1. Utility Types: DTO de Creación (sin ID) y DTO de Actualización Parcial
export type CrearVehiculoDto = Omit<VehiculoDto, 'id'>;
export type ActualizarVehiculoDto = Partial<CrearVehiculoDto>;

// 2. Interface Genérica para Repositorios Cliente
export interface RepositorioLectura<T, ID> {
  buscarPorId(id: ID): Promise<T>;
  listarTodos(): Promise<T[]>;
}

// 3. Predicado de Tipo (Type Guard Personalizado)
export function esVehiculo(objeto: any): objeto is VehiculoDto {
  return objeto && typeof objeto.placa === 'string' && typeof objeto.id === 'number';
}
```

```

}

// 4. Servicio Genérico de Cliente REST
export class ClienteRestGenerico<T extends { id: number }>
    implements RepositorioLectura<T, number> {

    constructor(private endpointBase: string) {}

    async buscarPorId(id: number): Promise<T> {
        const res = await fetch(`${this.endpointBase}/${id}`);
        if (!res.ok) throw new Error(`Registro ${id} no encontrado`);
        return await res.json();
    }

    async listarTodos(): Promise<T[]> {
        const res = await fetch(this.endpointBase);
        return await res.json();
    }

    // Uso de Partial<T> para peticiones HTTP PATCH
    async actualizarParcial(id: number, cambios: Partial<T>): Promise<T> {
        const res = await fetch(`${this.endpointBase}/${id}`, {
            method: 'PATCH',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(cambios)
        });
        return await res.json();
    }
}

```

Explicación Paso a Paso del Código:

1. **Omit<VehiculoDto, 'id'>:** Genera un tipo idéntico a `VehiculoDto` eliminando únicamente la propiedad `id`, perfecto para formularios de creación donde la base de datos asigna el ID.
2. **Partial<T>:** Permite enviar a `actualizarParcial` únicamente los campos que cambiaron (ejemplo: { capacidadKg: 6000 }), garantizando que los demás campos sean opcionales.
3. **objeto is VehiculoDto:** Predicado de tipo que le confirma a TypeScript en tiempo de compilación que si la función retorna `true`, la variable `objeto` debe tratarse como `VehiculoDto`.
4. **T extends { id: number }:** Restricción genérica que exige que cualquier entidad utilizada en `ClienteRestGenerico` posea obligatoriamente una propiedad numérica `id`.

Trampas Comunes

Uso de Aserciones de Tipo Forzadas (as Type / Type Casting Inseguro):

- *Error:* Hacer `const dato = respuesta as VehiculoDto`; sin verificar si los datos recibidos del servidor cumplen con la estructura, provocando fallas silenciosas en runtime.
- *Solución:* Usar *Type Guards* (`is`) o librerías de validación de esquemas (como Zod) en lugar de aserciones `as`.

Abuso de Genéricos Innecesarios:

- *Error:* Crear funciones con múltiples parámetros genéricos `<T, U, V>` cuando una interface o tipo de unión simple resolvería el problema de forma más legible.
- *Solución:* Utilizar genéricos únicamente cuando se requiera establecer relaciones de tipo entre parámetros de entrada y valores de retorno.

Confundir `Omit` con `Pick`:

- *Error:* Usar `omit` para seleccionar 2 propiedades de un objeto de 20 campos en lugar de usar `Pick`.
- *Solución:* Usar `Pick` cuando se desea seleccionar un grupo pequeño de campos y `omit` cuando se desea descartar sólo uno o dos campos.

Pregunta de Reflexión

En la arquitectura del framework Angular:

¿De qué manera la combinación de **Genéricos** (en servicios `HttpClient.get<T>()`), **Utility Types** (en el manejo de formularios reactivos) y **Decoradores** (`@Component`, `@Injectable`) permite construir aplicaciones empresariales escalables con autocompletado y detección de errores en tiempo de

compilación?